

The Perceptron

CSE 5526 - Intro to Neural Networks
Lecture 5

Instructor: Chris Brew

Psychological Review
Vol. 65, No. 6, 1958

THE PERCEPTRON: A PROBABILISTIC MODEL FOR INFORMATION STORAGE AND ORGANIZATION IN THE BRAIN¹

F. ROSENBLATT

Cornell Aeronautical Laboratory

If we are eventually to understand the capability of higher organisms for perceptual recognition, generalization, recall, and thinking, we must first have answers to three fundamental questions:

1. How is information about the physical world sensed, or detected, by the biological system?

2. In what form is information

and the stored pattern. According to this hypothesis, if one understood the code or "wiring diagram" of the nervous system, one should, in principle, be able to discover exactly what an organism remembers by reconstructing the original sensory patterns from the "memory traces" which they have left, much as we might develop a photographic negative, or translate

Agenda and Learning Outcomes

Today's Topic(s)

- **Goal(s):**
 - Recap perceptrons, decision boundaries, need for multi-layer perceptrons
 - Describe and illustrate the Perceptron Learning Rule
 - Python perceptron example
 - Introduce the Perceptron Convergence Theorem

Recall: Perceptron Neuron Model

Modified version of McCulloch-Pitts neuron

- **Perceptrons** allow *real-valued inputs*. Everything else is the same
- Invented by Rosenblatt in 1957
- Useful for binary classification (e.g. only two different classes of data)
 - Neuron outputs 1 when a specific class is encountered
 - It outputs -1 when the other class is encountered
- **Perceptrons are also used for learning, unlike McCulloch-Pitts neurons**

$$x_i \in \mathbb{R}$$

Real-valued inputs

$$v = \sum_{i=1}^m w_i x_i + b$$

Activation potential

$$y = \phi(v)$$

Output

$$\phi(v) = \begin{cases} 1, & \text{if } v \geq 0 \\ -1, & \text{if } v < 0 \end{cases}$$

Activation function

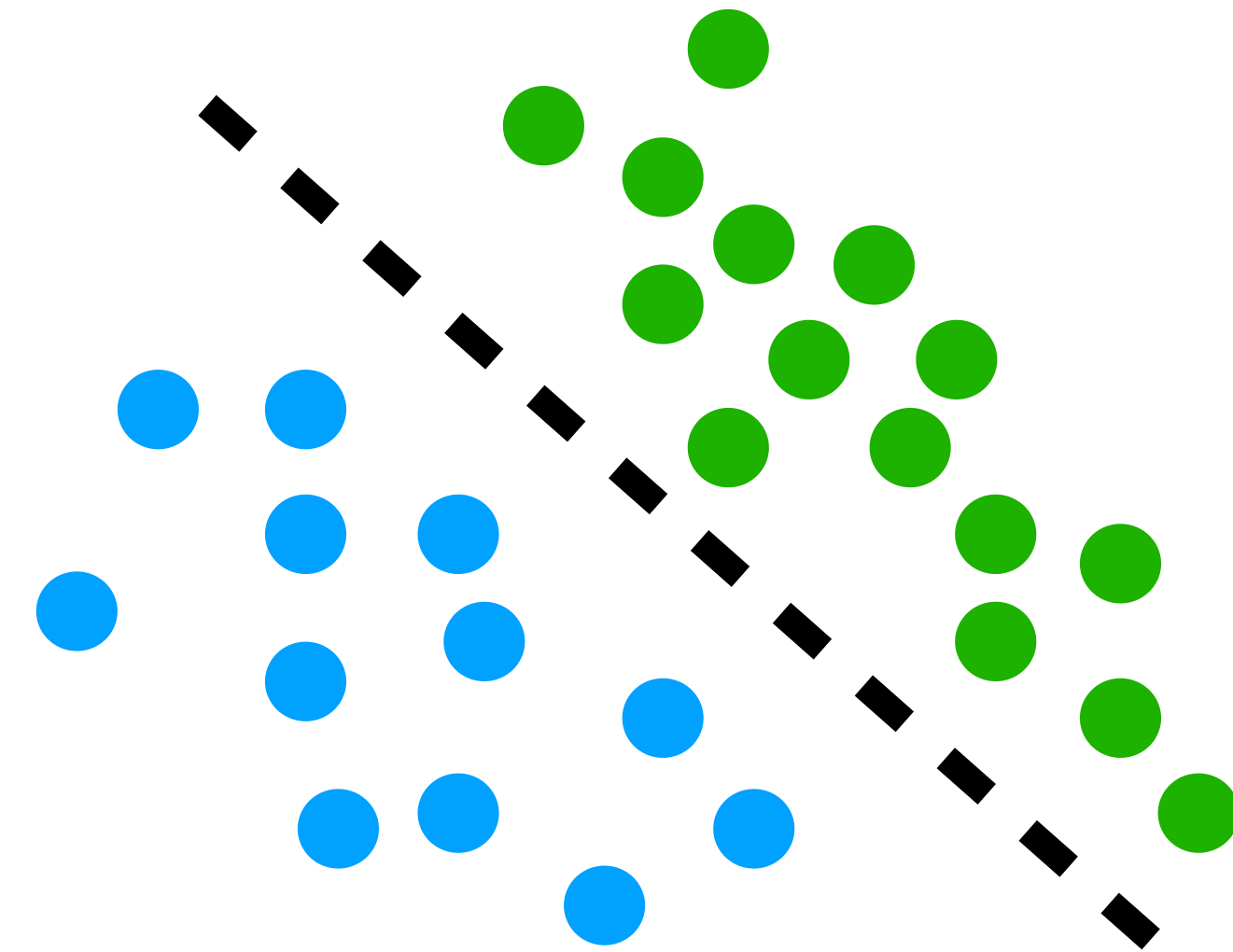
Recall: Perceptrons for Classification

Decision Boundary

- The ***decision boundary of a perceptron*** for a given $\mathbf{w}$:

$$g(\mathbf{x}) = \mathbf{w}^T \mathbf{x} = \sum_{i=0}^m w_i x_i = 0$$

- $g(\mathbf{x})$ is also called the ***discriminant function*** for the perceptron, and it is a linear function of $\mathbf{x}$. Hence, it is a *linear discriminant function*



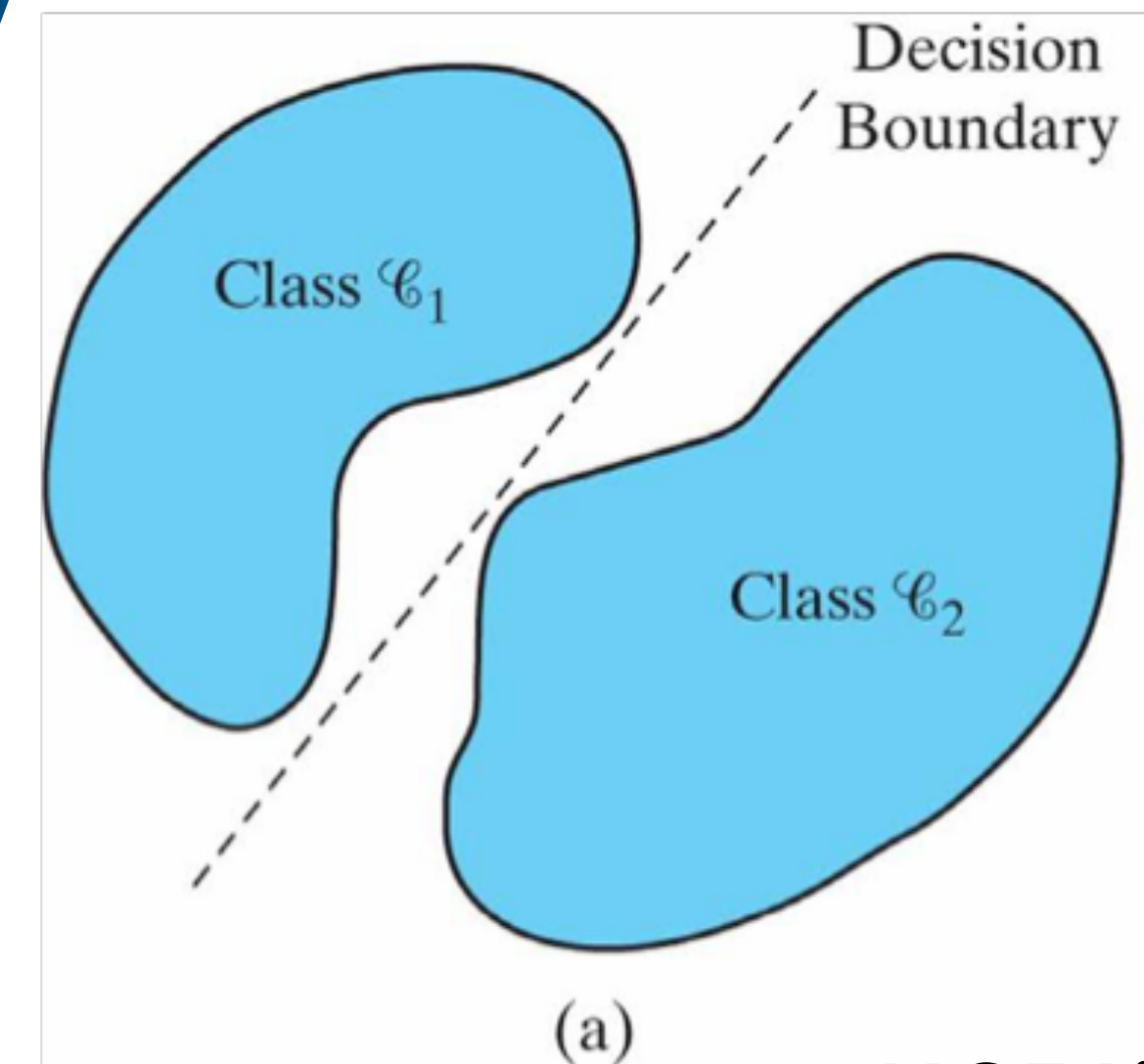
Why is the decision boundary set to 0?

Recall: Linear Separability

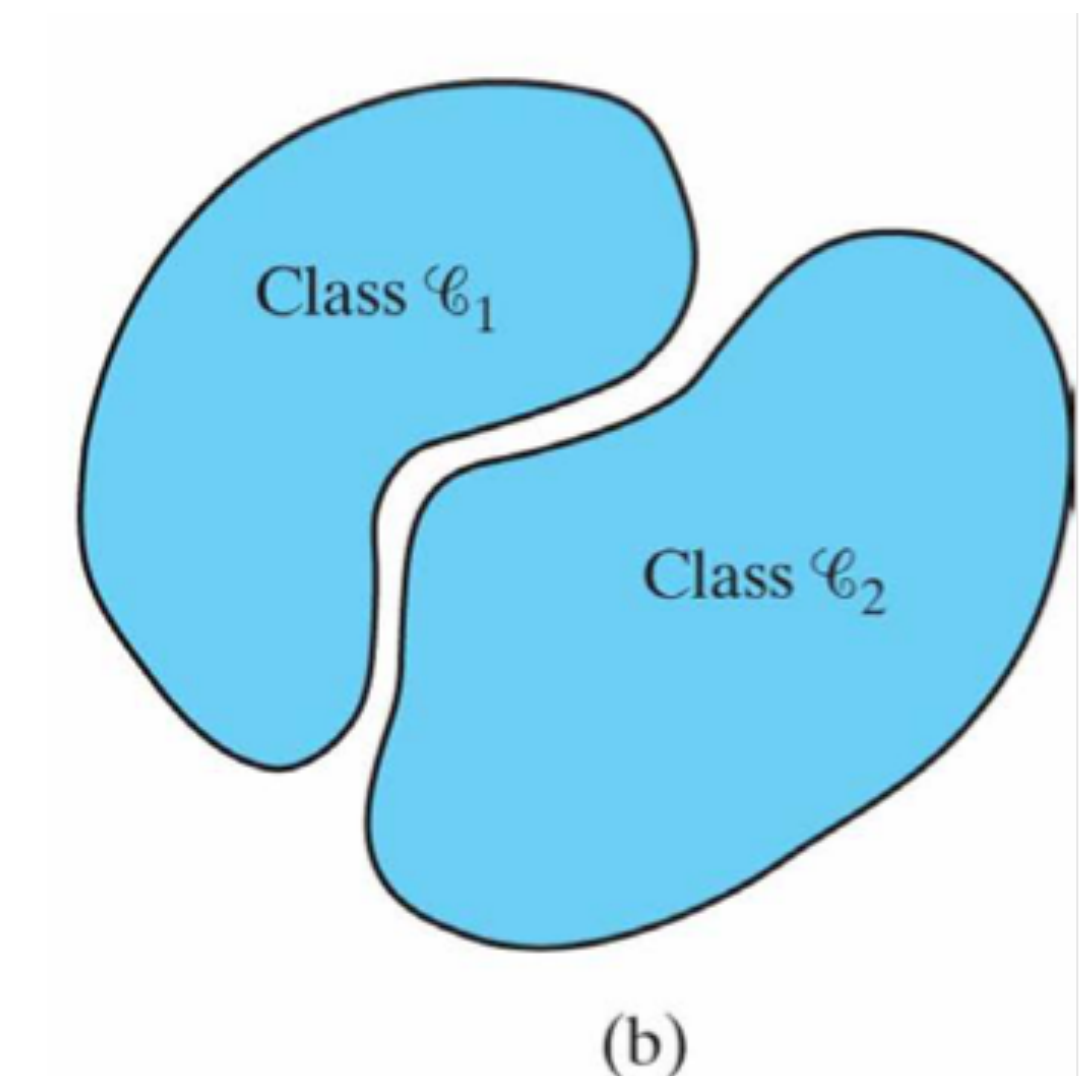
Perceptrons

- For the set of input patterns $\mathbf{x}$, if there exists one $\mathbf{w}$ that separates $d = 1$ (e.g. class A) patterns from $d = -1$ (class B) patterns, then the classification is called ***linearly separable***
- In other words, there exists a linear discriminant function that produces no classification error
- This is a very important concept!

Linearly Separable



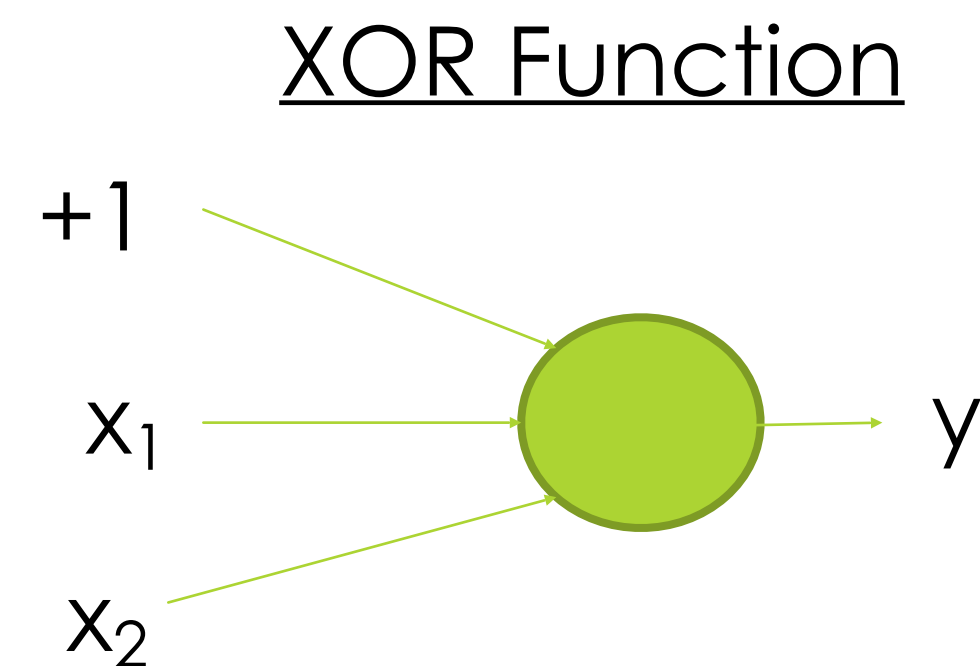
NOT Linearly Separable



A limitation of single-layer perceptrons

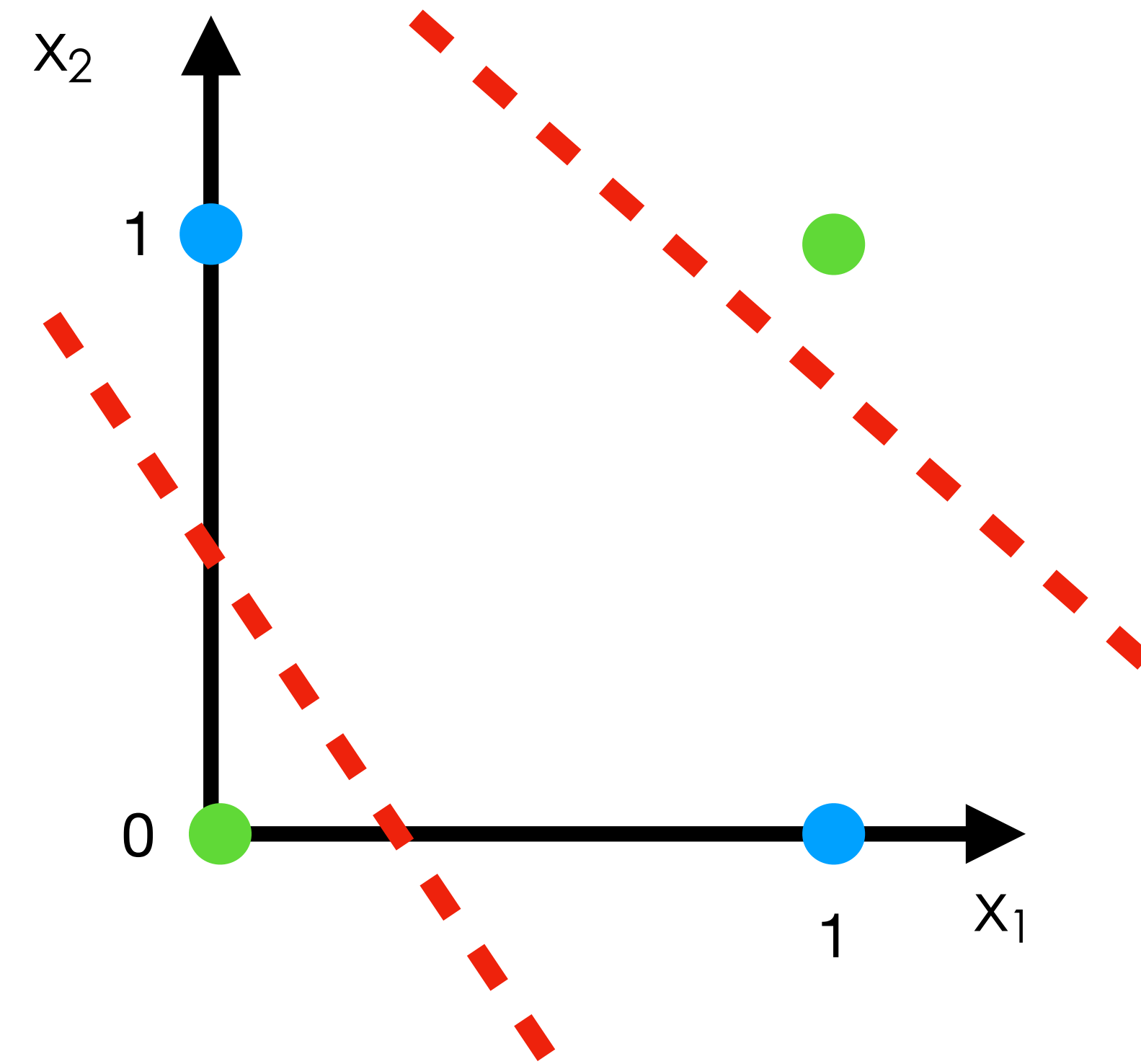
Logic Gate Example: XOR Gate

Single-layer perceptron networks, however, are not useful for linearly inseparable problems or linearly-separable problems that require multiple boundaries per class



x1	x2	d
0	0	0
0	1	1
1	0	1
1	1	0

A single (layer) perceptron cannot solve this problem



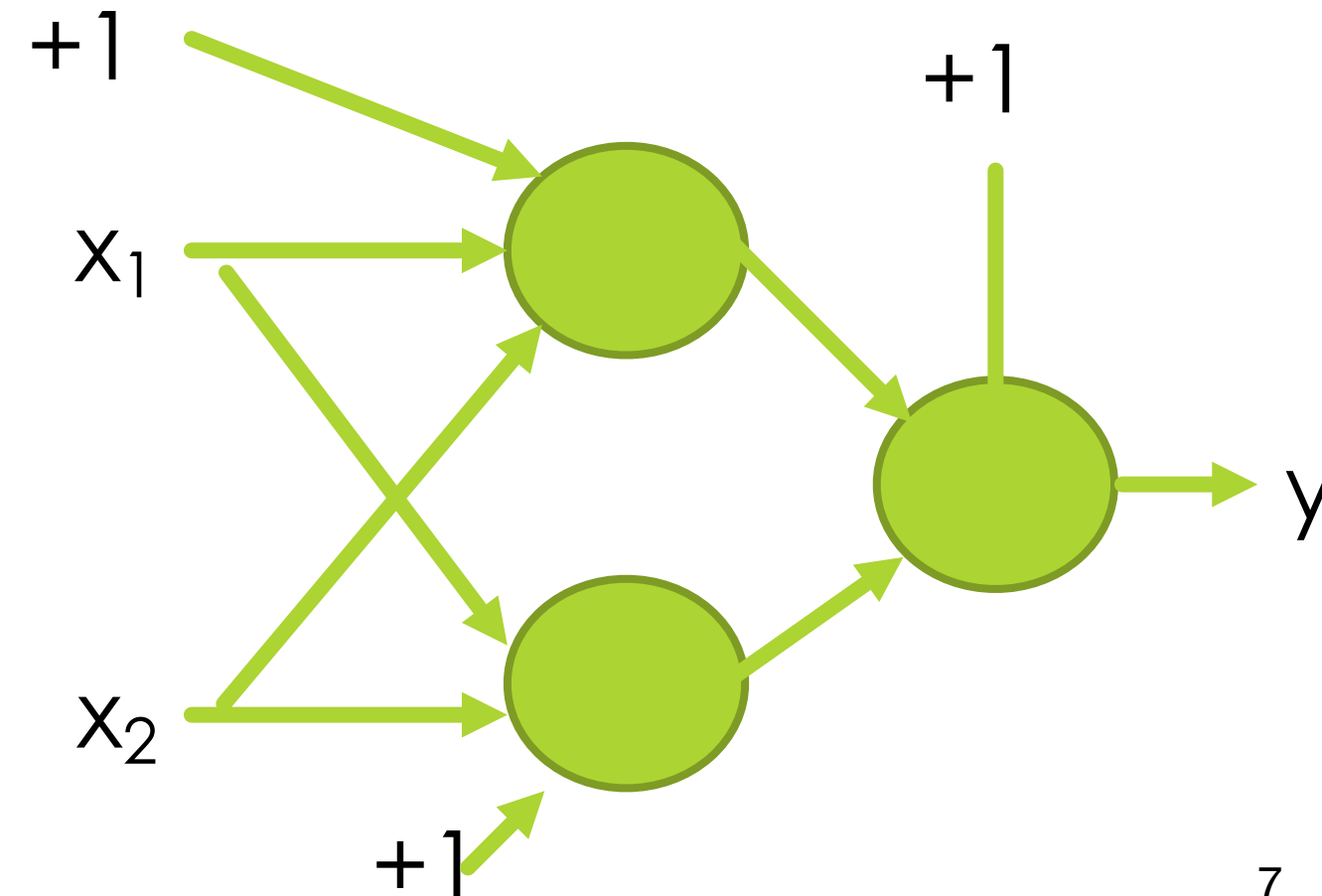
The Case for Multi-Layered Perceptrons (MLP)

Logic Gate Example

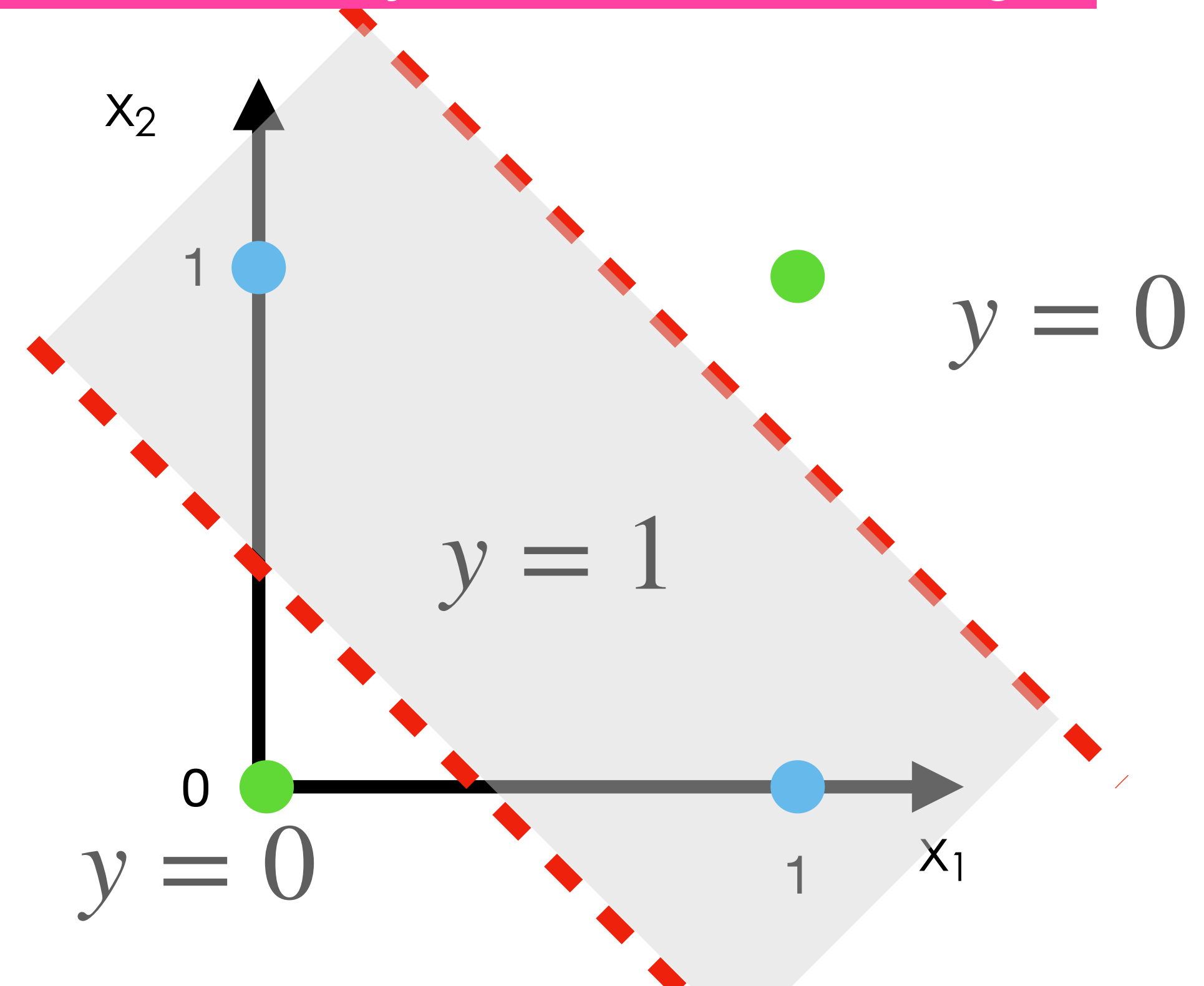
Adding layers to a network can help solve more complicated problems. The resulting neural network is called a **Multi-Layer Perceptron (MLP)**

XOR Function

x1	x2	d
0	0	0
0	1	1
1	0	1
1	1	0



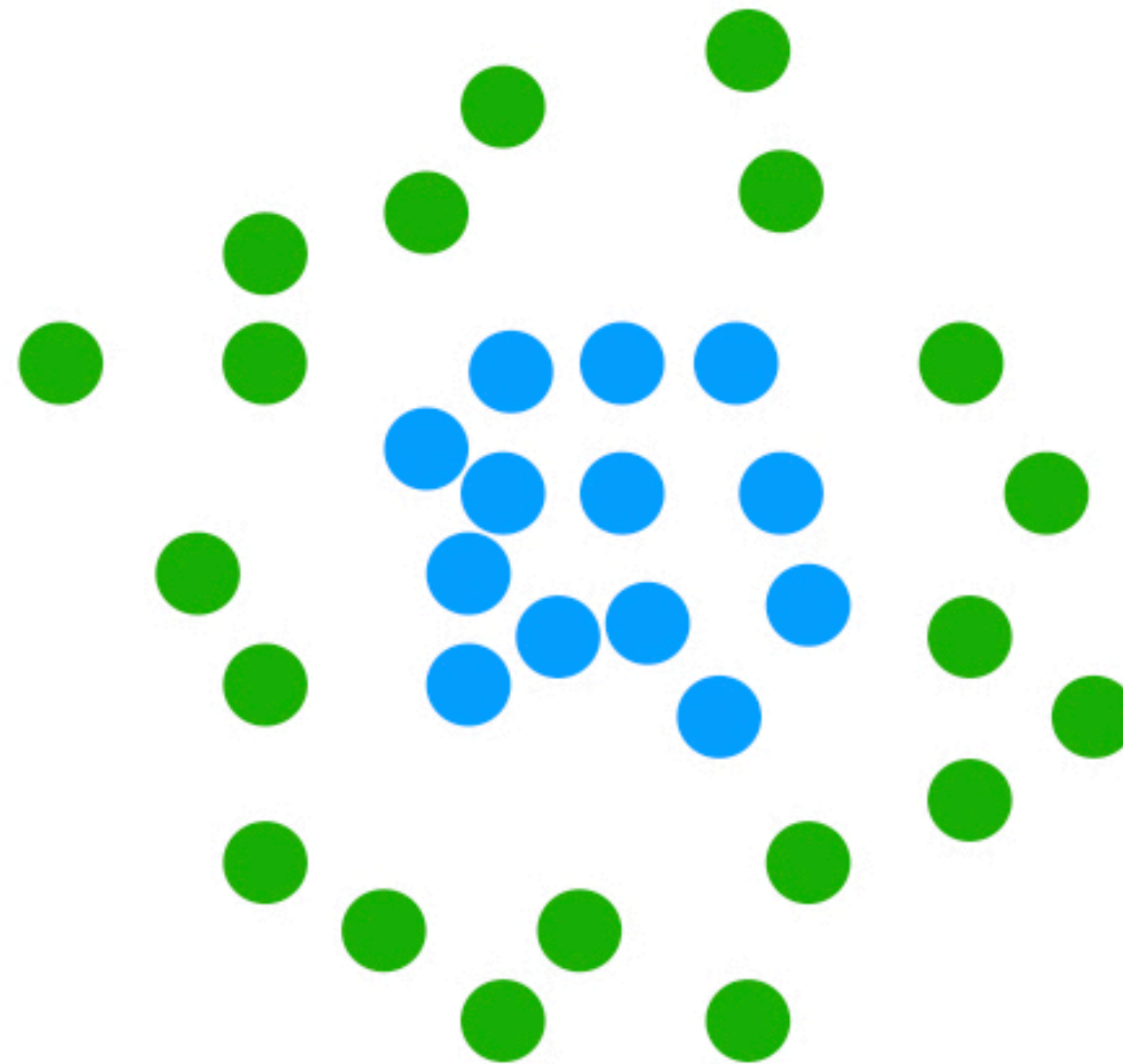
The decision boundary becomes a decision region



Linearly Inseparable Case

What needs to happen here?

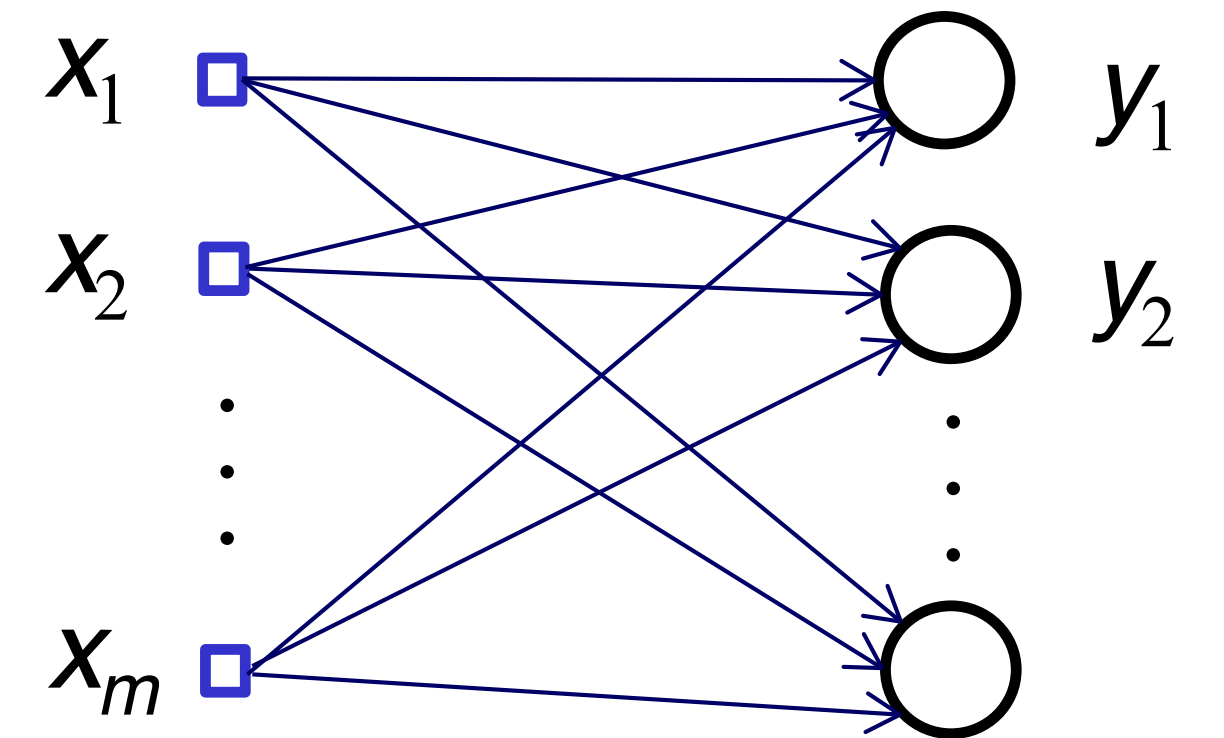
- How can this be solved with a multi-layer perceptron?



Questions?

Perceptrons for Multi-class classification

- Multiple perceptrons can be used when performing multi-class classification (one for each class)
- Perceptron output is 1 when input is from the corresponding class
- It's output is -1 otherwise
- Linear separability must apply between classes

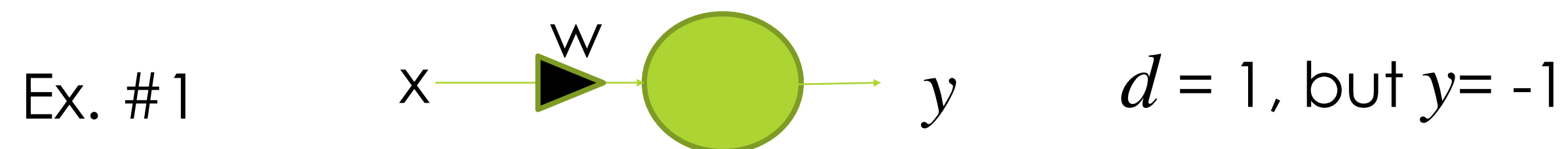


Questions?

Perceptron Learning Rule

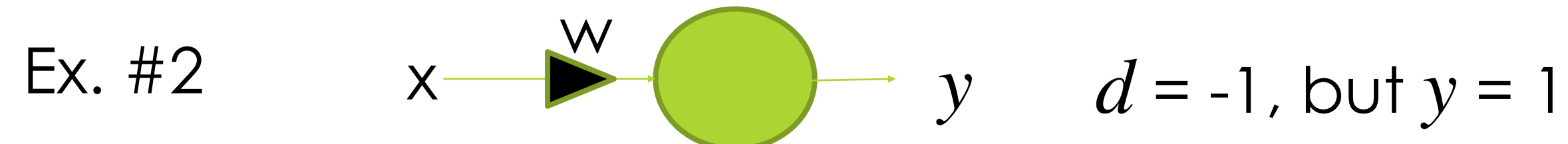
Finding weights that separate classes

- For linearly separable problems, our goal is to “learn” a set of weights that results in zero classification errors
- **Perceptron Learning Rule (Rosenblatt):**
 - **Strengthen the weight** if the neuron fails to fire when it should have fired
 - **Weaken the weight** if the neuron fires when it should not have fired
- **Example:** Single perceptron with single input



If want y to be 1, but the output is -1, then this neuron should be strengthened.

In other words, the weight is too small. It needs to increase



This neuron should be weakened. The weight should decrease.

Perceptron Learning Rule

Single Perceptron, Multiple inputs

- How do we change the weights if they are too small or too large?

$$\mathbf{w}(n + 1) = \mathbf{w}(n) + \Delta(\mathbf{w})$$

n : iteration number

$$= \mathbf{w}(n) + \eta[d(n) - y(n)]\mathbf{x}(n)$$

η : step size or learning rate

Actual output

Desired output

- Note:
 - The outputs are bipolar $\{-1, 1\}$
 - What are the possible values for $[d-y]$?

Perceptron Learning Rule

Single Perceptron, Multiple inputs

- How do we change the weights if they are too small or too large?

$$\mathbf{w}(n + 1) = \mathbf{w}(n) + \Delta(\mathbf{w})$$

n : iteration number

$$= \mathbf{w}(n) + \eta[d(n) - y(n)]\mathbf{x}(n)$$

η : step size or learning rate

Actual output

Desired output

- Note:

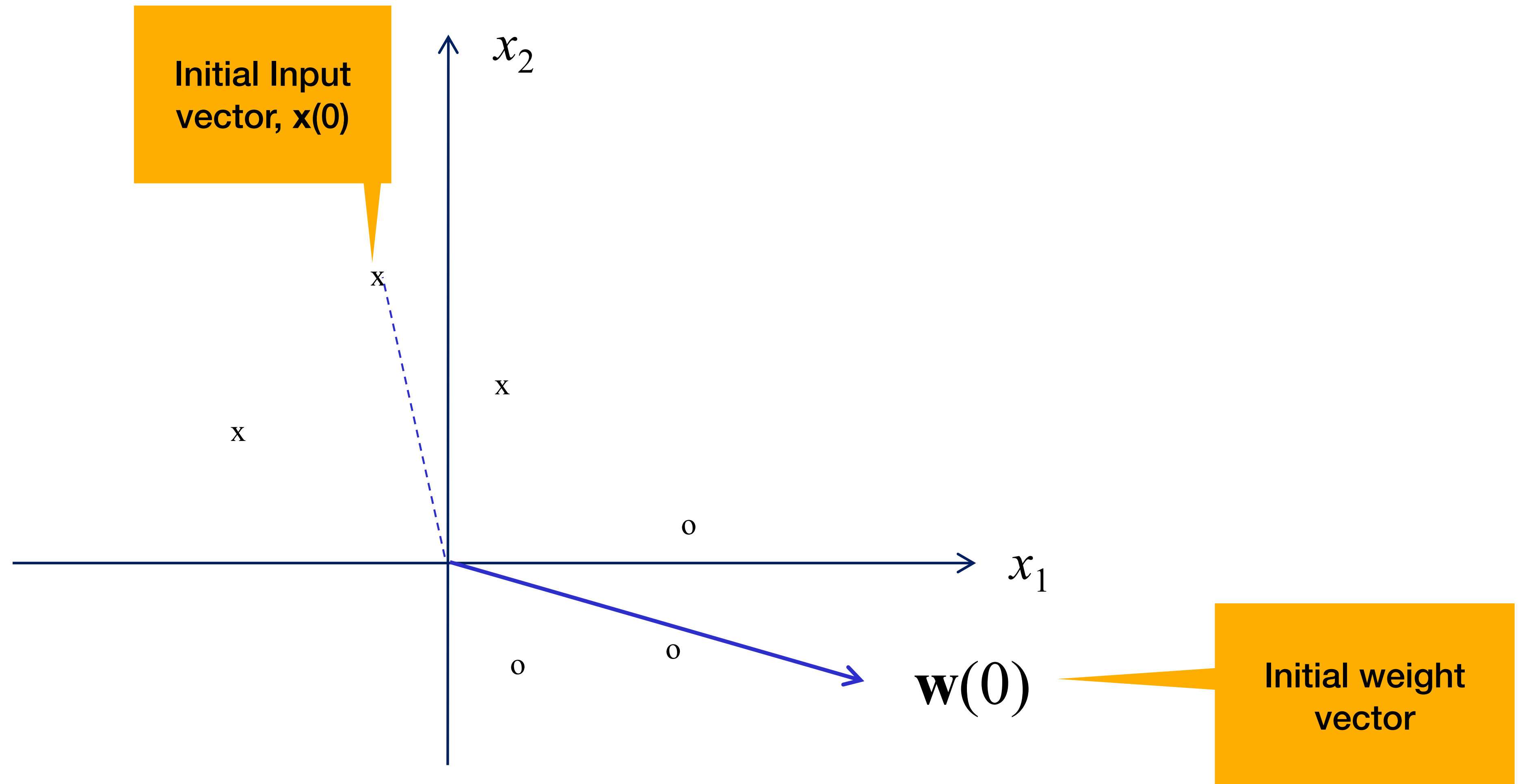
- The outputs are bipolar $\{-1, 1\}$
- What are the possible values for $[d-y]$?
 - Thus, $[d-y]$ is either 0 (correct), -2 (too strong), or 2 (too weak)

Perceptron Learning Rule

A Geometric Interpretation - 2D Case

- Assume $\eta = \frac{1}{2}$
- $d = 1$ for x
- $d = -1$ for o

- If the angle between the weight vector and the input vector is less than (or equal to) 90° , then the output is 1
 - If it is greater than 90° , then the output is -1
- $$\mathbf{w}(n+1) = \mathbf{w}(n) + \eta[d(n) - y(n)]\mathbf{x}(n)$$

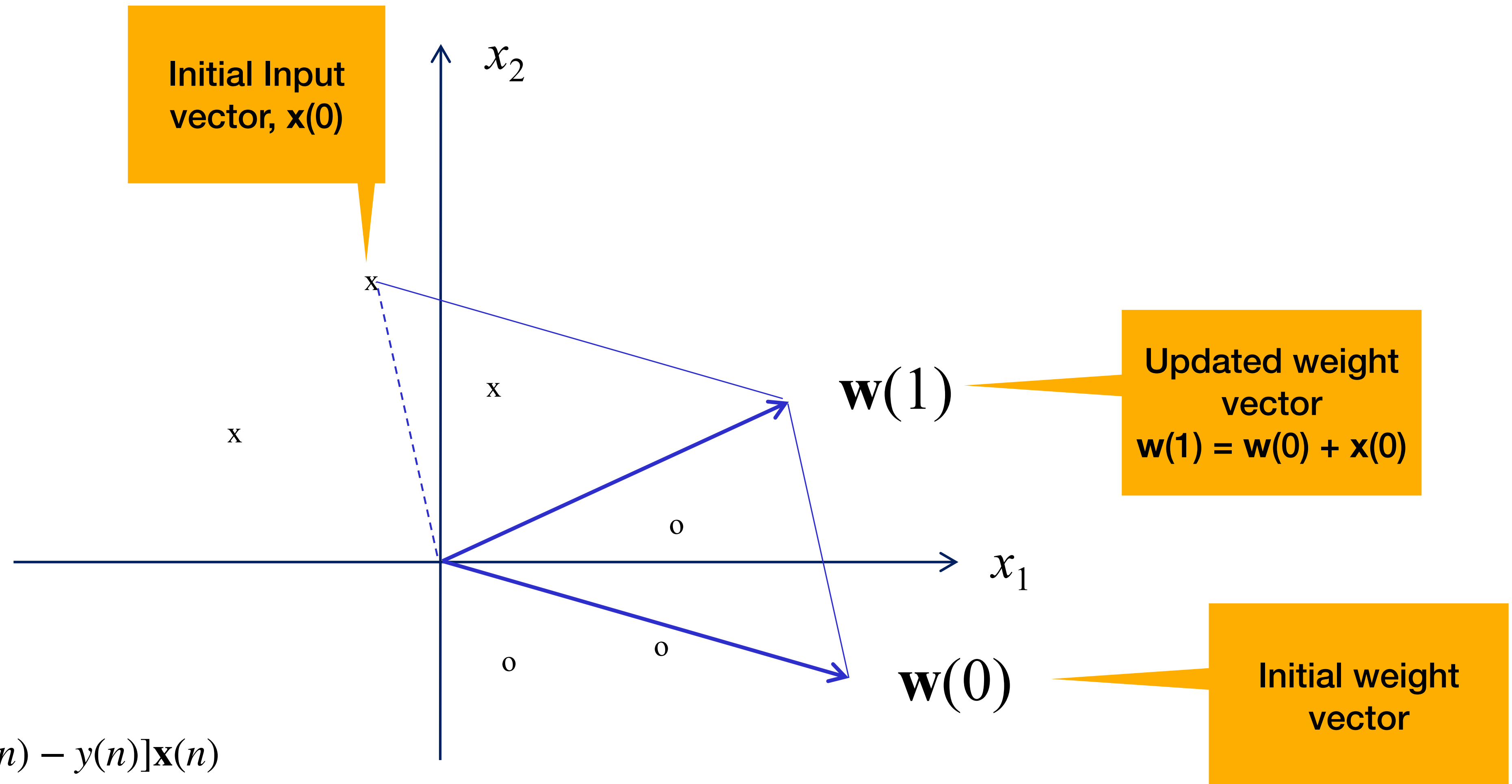


Perceptron Learning Rule

A Geometric Interpretation - 2D Case

- If the angle between the weight vector and the input vector is less than (or equal to) 90° , then the output is 1
- If it is greater than 90° , then the output is -1

- Assume $\eta = \frac{1}{2}$
- $d = 1$ for x
- $d = -1$ for o

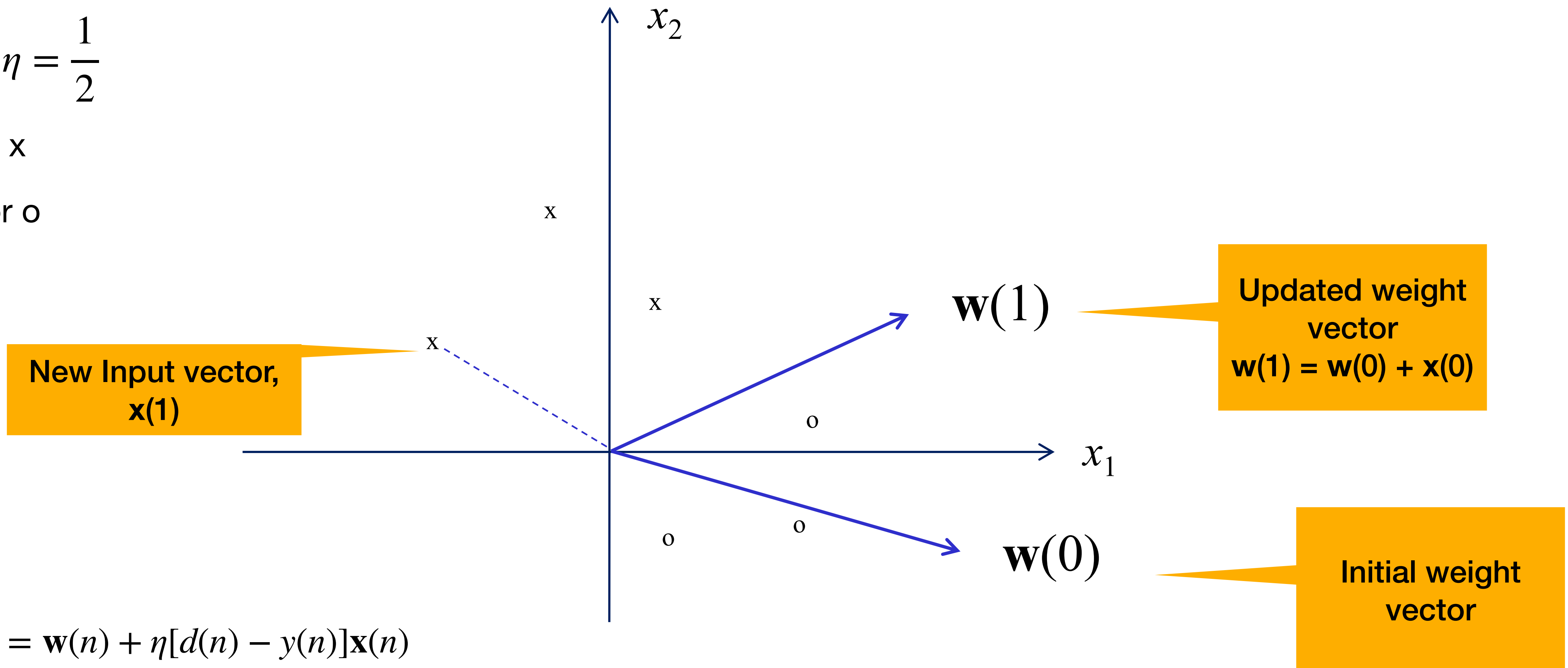


Perceptron Learning Rule

A Geometric Interpretation - 2D Case

- If the angle between the weight vector and the input vector is less than (or equal to) 90° , then the output is 1
- If it is greater than 90° , then the output is -1

- Assume $\eta = \frac{1}{2}$
- $d = 1$ for x
- $d = -1$ for o

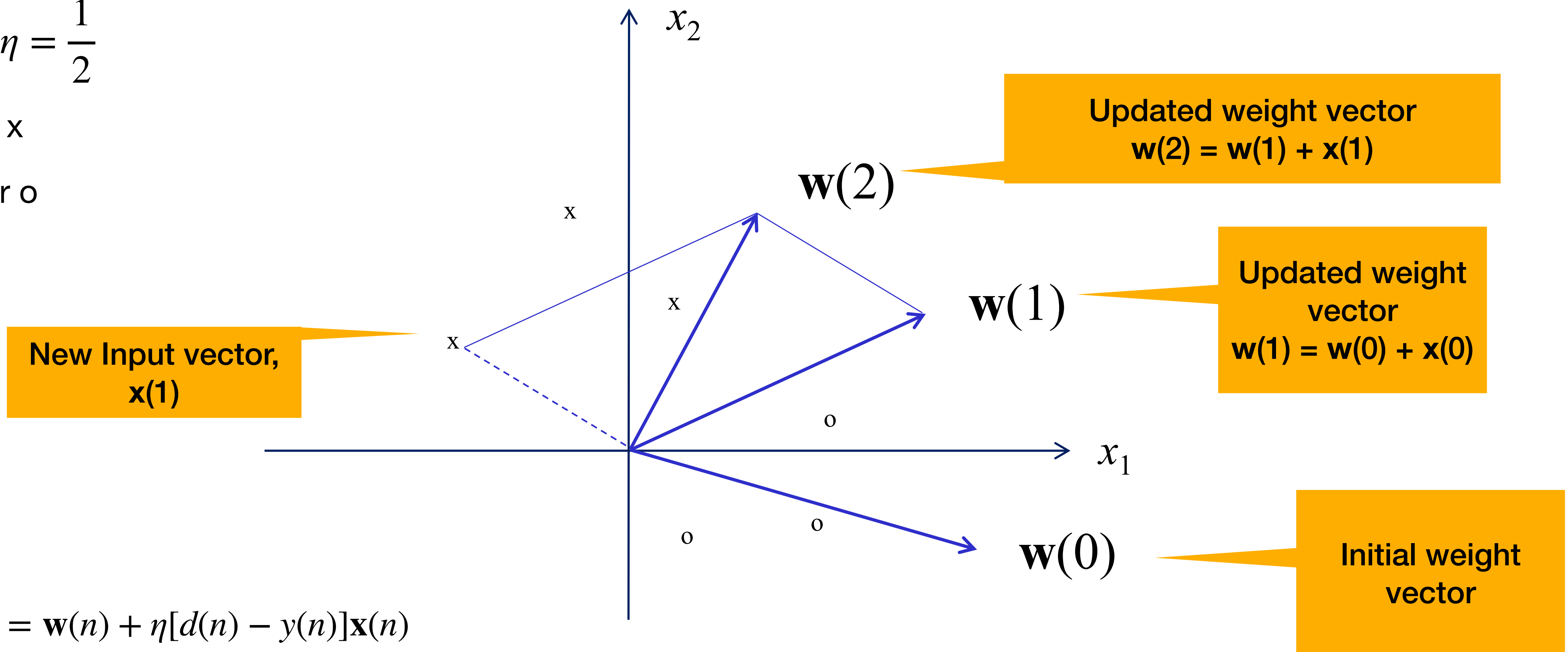


Perceptron Learning Rule

A Geometric Interpretation - 2D Case

- If the angle between the weight vector and the input vector is less than (or equal to) 90° , then the output is 1
- If it is greater than 90° , then the output is -1

- Assume $\eta = \frac{1}{2}$
- $d = 1$ for x
- $d = -1$ for o



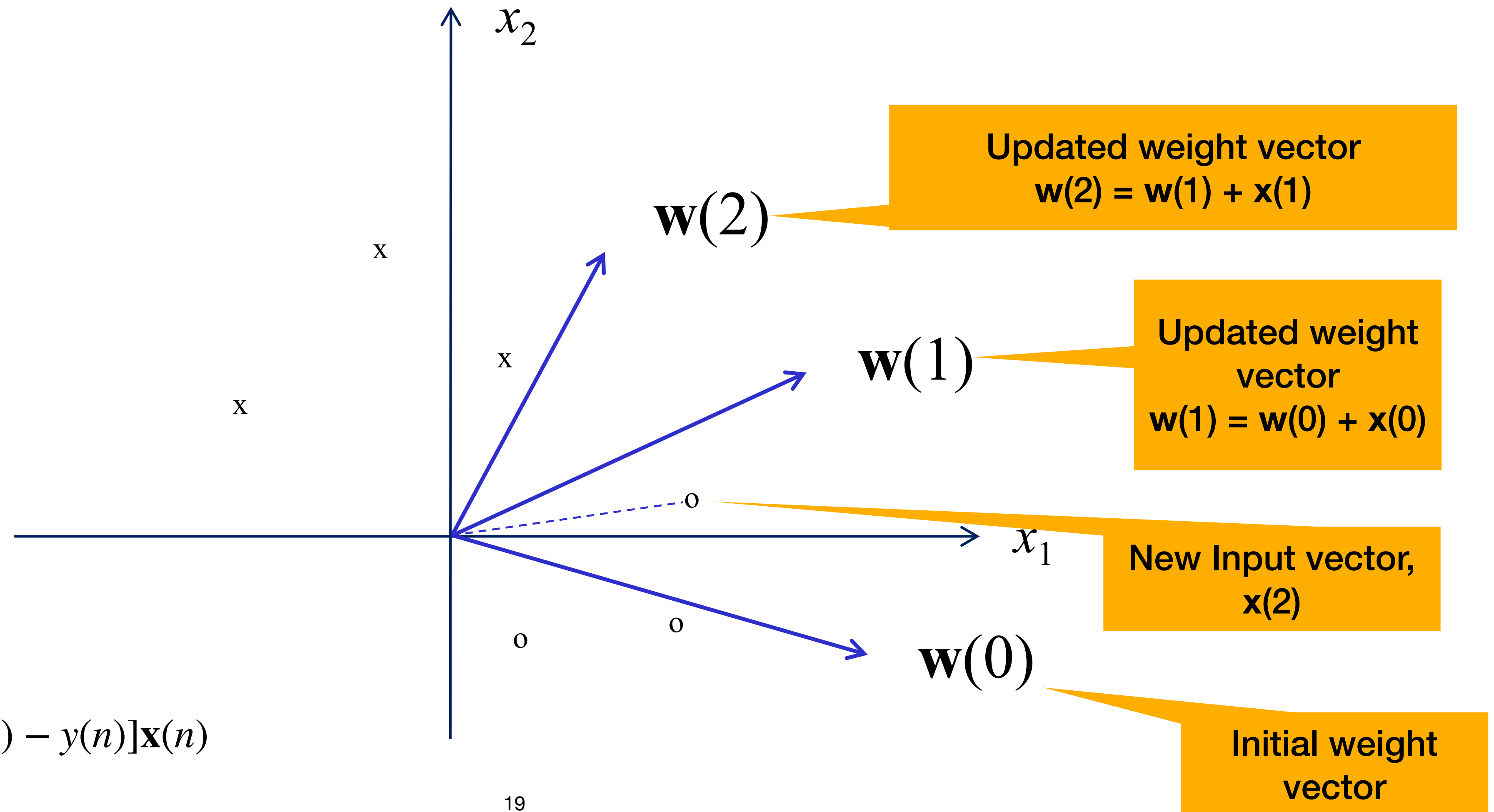
$$\mathbf{w}(n + 1) = \mathbf{w}(n) + \eta[d(n) - y(n)]\mathbf{x}(n)$$

Perceptron Learning Rule

A Geometric Interpretation - 2D Case

- If the angle between the weight vector and the input vector is less than (or equal to) 90° , then the output is 1
- If it is greater than 90° , then the output is -1

- Assume $\eta = \frac{1}{2}$
- $d = 1$ for x
- $d = -1$ for o



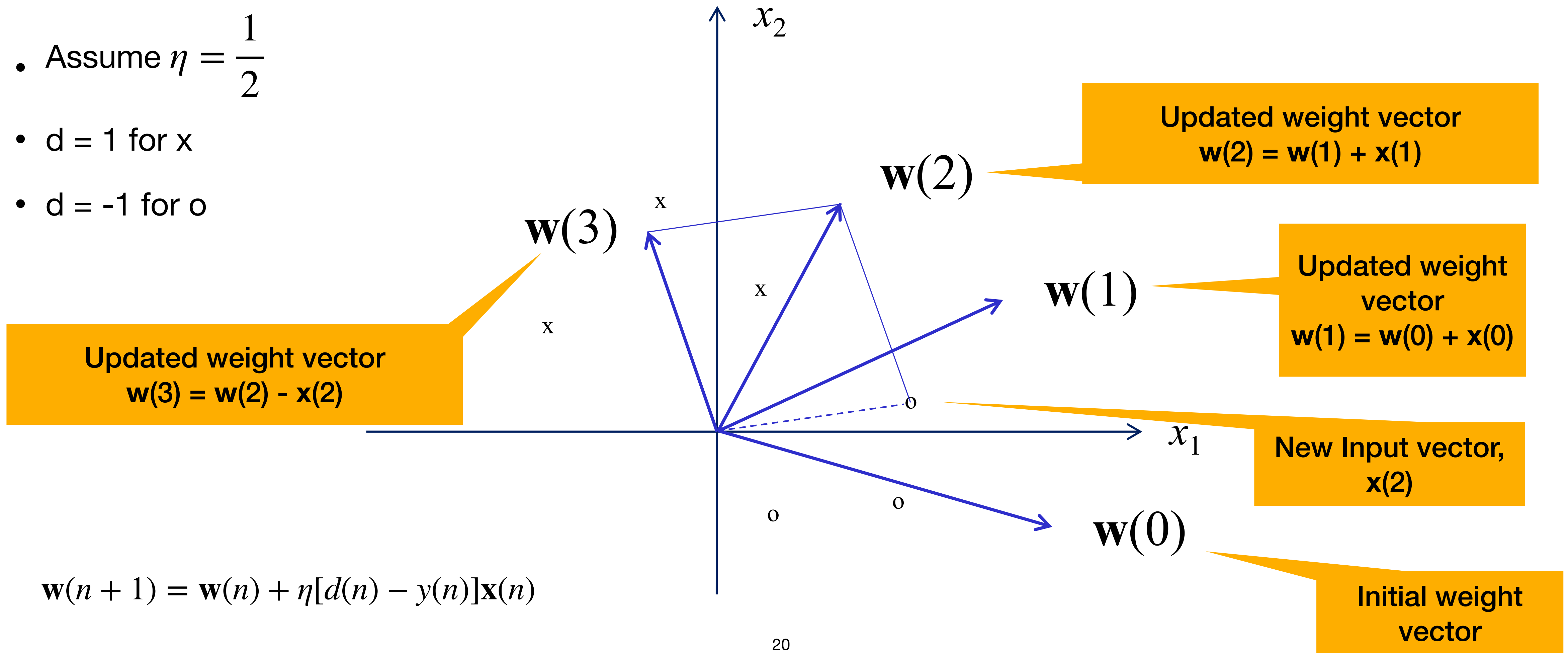
$$w(n + 1) = w(n) + \eta[d(n) - y(n)]x(n)$$

Perceptron Learning Rule

A Geometric Interpretation - 2D Case

- If the angle between the weight vector and the input vector is less than (or equal to) 90° , then the output is 1
- If it is greater than 90° , then the output is -1

- Assume $\eta = \frac{1}{2}$
- $d = 1$ for x
- $d = -1$ for o

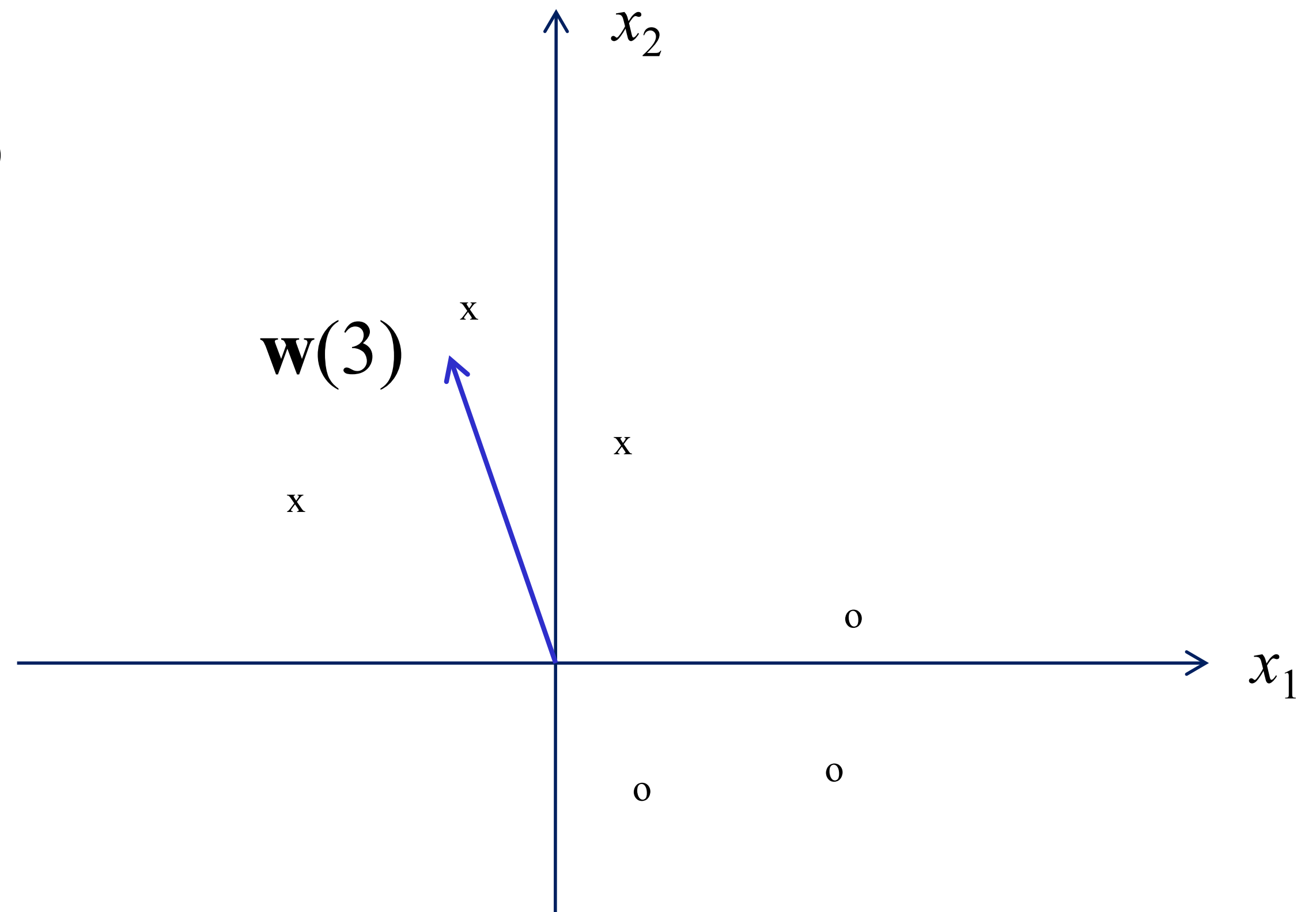


Perceptron Learning Rule

A Geometric Interpretation - 2D Case

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \eta[d(n) - y(n)]\mathbf{x}(n)$$

- Each weight update moves $\mathbf{w}$ closer to $d=1$ patterns or away from $d = -1$ patterns
- $\mathbf{w}(3)$ solves the classification problem for all input patterns
 - Check on your own (e.g. the angles)



Questions?

Perceptron for Flower Classification

A Python Example

- There are three classes of Iris flowers: Virginica, Setosa, Versicolor
- Let's classify if the flower is a Iris-Virginica ($d = 1$) or not, based on the petal length and width



Import libraries

```
: import numpy as np
   from sklearn import datasets
   from sklearn.linear_model import Perceptron
   from matplotlib import pyplot as plt
```

Load Data

```
: iris = datasets.load_iris()
   list(iris.keys())

: ['data',
   'target',
   'frame',
   'target_names',
   'DESCR',
   'feature_names',
   'filename',
   'data_module']
```


Perceptron for Flower Classification

A Python Example: Feature Extraction

- There are three classes of Iris flowers: Virginica, Setosa, Versicolor
- Let's classify if the flower is a Iris-Virginica ($d = 1$) or not, based on the petal length and width



Extract features and labels

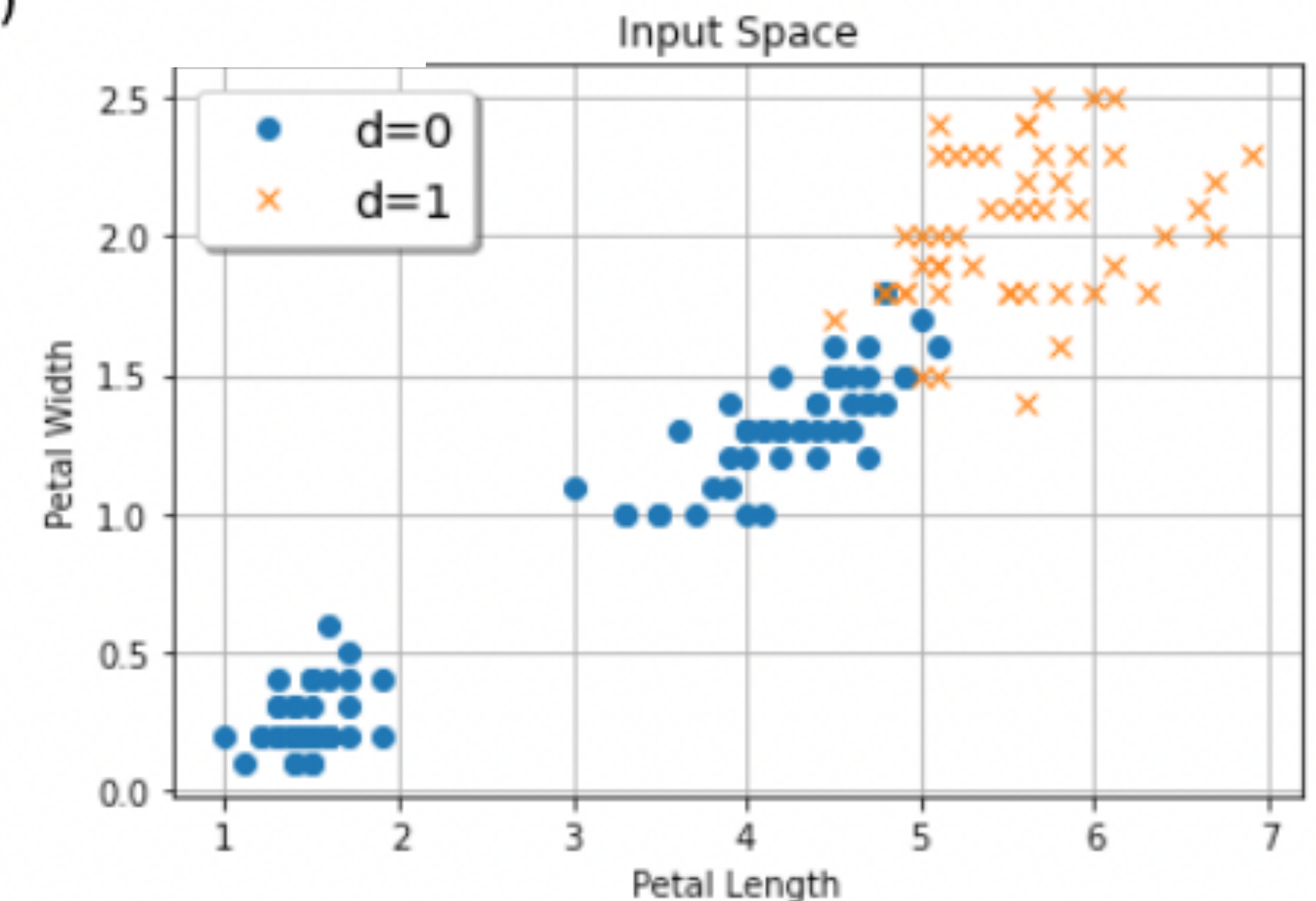
```
X = iris["data"][:,(2,3)] # petal length/width  
d = (iris["target"] == 2).astype(int) # Iris-Virginica
```

```
print(X.shape)
```

```
(150, 2)
```

```
print(d.shape)
```

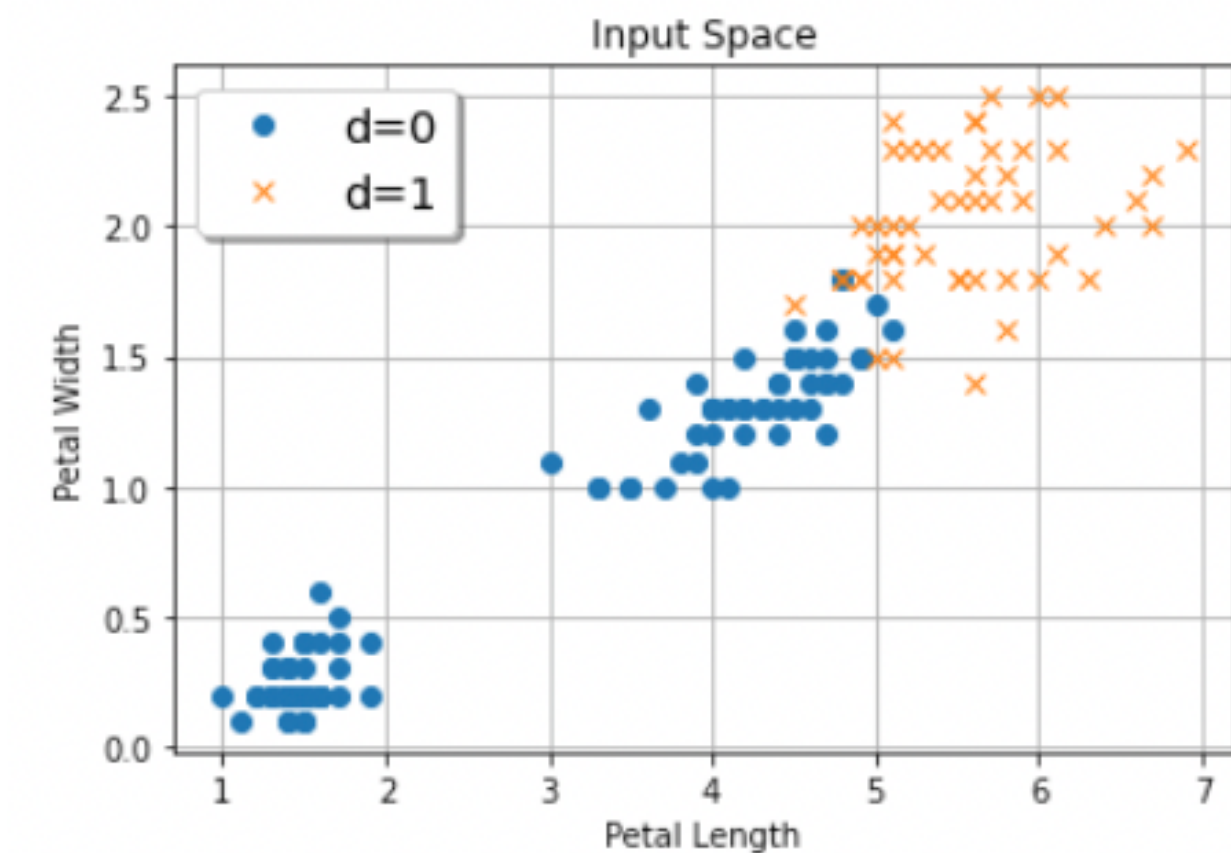
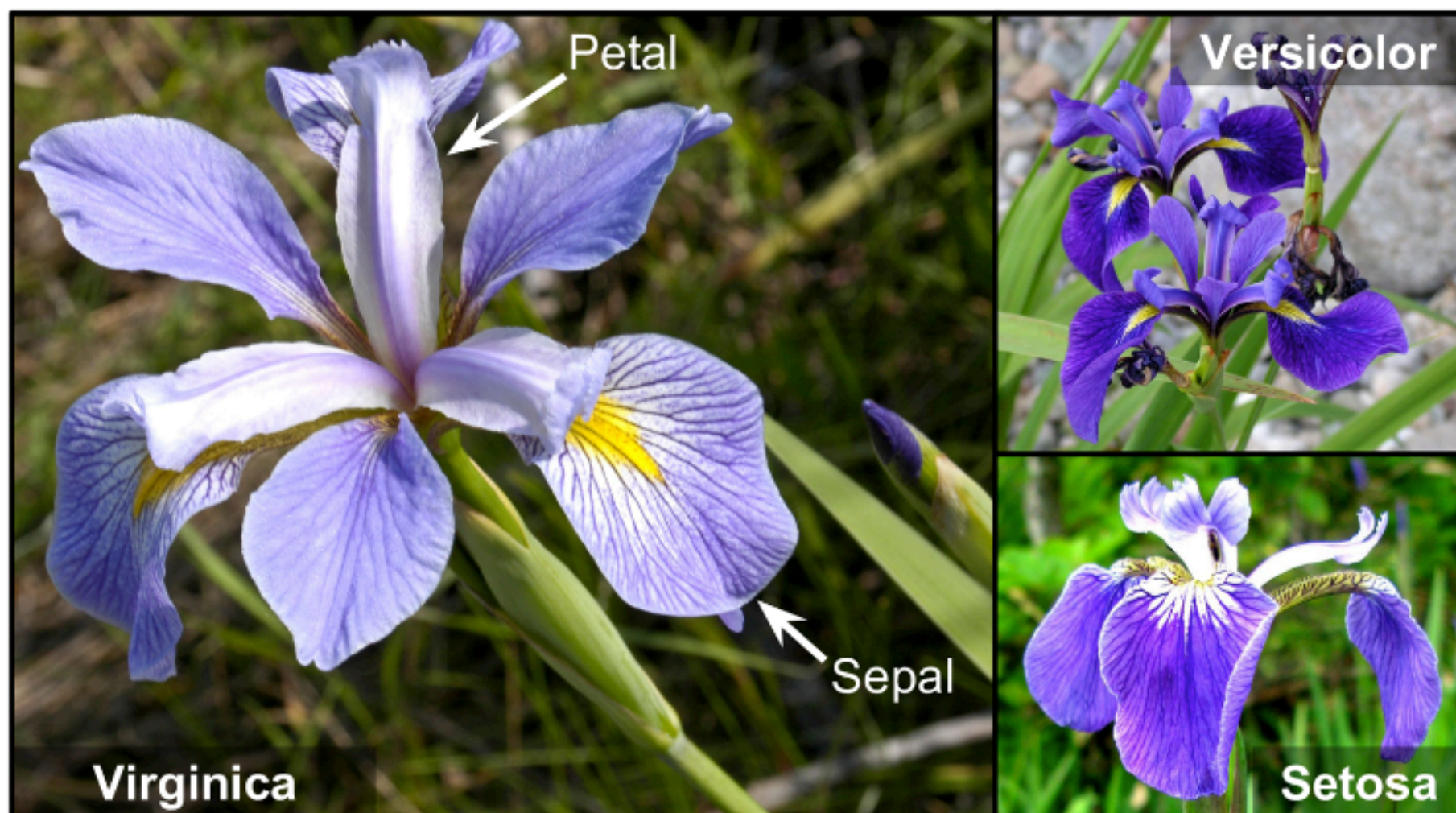
```
(150,)
```



Perceptron for Flower Classification

A Python Example: Train and Evaluate Perceptron

- There are three classes of Iris flowers: Virginica, Setosa, Versicolor
- Let's classify if the flower is a Iris-Virginica ($d = 1$) or not, based on the petal length and width



Initialize and train perceptron

```
per_clf = Perceptron(random_state = 42)
per_clf.fit(X,d)
```

```
Perceptron(random_state=42)
```

Get classifier results on data

```
per_clf.score(X,d) #accuracy
```

```
0.7266666666666667
```

Questions?

Perceptron Convergence Theorem

- **Theorem:** If a classification problem is linearly separable, a perceptron will reach a solution in a *finite* number of iterations
 - In other words, a perceptron can be trained to solve all linearly separable problems
 - This is important!
- **Why does this matter?**
 - You can solve the problem with a perceptron if the data is linearly separable
 - May also solve it with a perceptron if the data can be modified to become linearly separable

Result of Proof

Perceptron Convergence Theorem

- Can show that the number of steps, P , has to be finite:

$$\frac{\alpha^2 P}{\beta \|\mathbf{w}_o\|^2} \leq \frac{(\mathbf{w}_o^T \mathbf{w})^2}{\|\mathbf{w}_o\|^2 \|\mathbf{w}\|^2} \leq 1$$

Proof is provided later, if you are interested.

where,

- P is the number of iterations
- $\mathbf{w}_o$ are the optimal weights that solve the problem
- $\mathbf{w}$ is the set of weights, after a single weight update
- β is the length of the longest training data sample
- α is

$$\alpha = \min_p d_p \mathbf{w}_o^T \mathbf{x}_p$$

$$\beta = \max_p \|\mathbf{x}_p\|^2$$

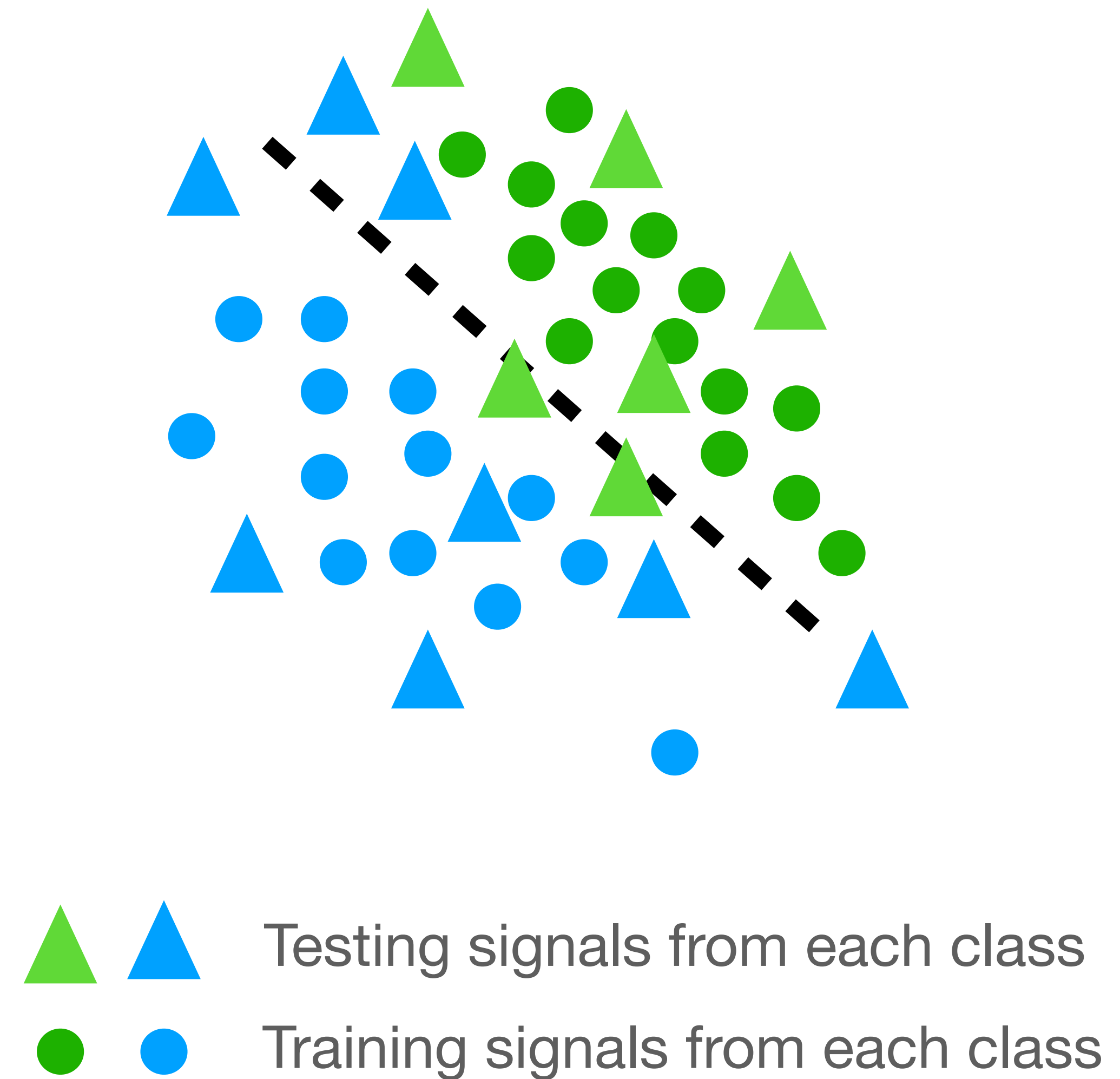
Remarks

- In the case of $\mathbf{w}(0) = \mathbf{0}$, the learning rate has no effect on the proof. That is, the theorem holds no matter what η ($\eta > 0$) is
 - It may affect the rate of convergence though (e.g. number of iterations)
- The solution weight vector is **not** unique
 - There are an infinite number of decision boundaries and solutions that can solve the problem
 - It depends on the order of the input patterns and how the weights are initialized

Generalization Once Trained

Perceptrons during testing

- **Generalization**: Performance of a learning machine on test patterns not seen (or used) during training.
- **Perceptrons generalize by deriving a decision boundary in the input space.**
 - The selection of training patterns is thus important for generalization
 - The solution weight vector is not unique. There are infinite possible solutions and decision boundaries.



Next class:

**Linear neurons, the Normal
Equation, and Gradient
Descent**



Proof of Perceptron Convergence Theorem

Perceptron Convergence Theorem

Proof: Existence of $\mathbf{w}_o$

- Given a finite number of training patterns, because of linear separability, there exists a weight vector $\mathbf{w}_o$ so that

$$d_p \mathbf{w}_o^T \mathbf{x}_p \geq \alpha > 0 \quad (1)$$

where

$$\alpha = \min_p d_p \mathbf{w}_o^T \mathbf{x}_p$$

Assigned the smallest product amongst all input patterns

If linearly separable, there is a set of weights where the weighted sum of inputs has the same sign as the desired output

Perceptron Convergence Theorem

Proof: Sign of desired output and the activation potential

- Given a finite number of training patterns, because of linear separability, there exists a weight vector $\mathbf{w}_o$ so that

$$d_p \mathbf{w}_o^T \mathbf{x}_p \geq \alpha > 0 \quad (1)$$

where

$$\alpha = \min_p d_p \mathbf{w}_o^T \mathbf{x}_p$$

Perceptron Convergence Theorem

Proof: finishing correctness of inequality

- The classification is correct when the weighted sum of the input has the same sign as the desired output

$$d_p \mathbf{w}_o^T \mathbf{x}_p \geq \alpha > 0 \quad (1)$$

$$\alpha = \min_p d_p \mathbf{w}_o^T \mathbf{x}_p$$

Proof (cont.)

Perceptron Convergence Theorem

- We assume that the initial weights are all zero
- Let N_p denote the number of times $\mathbf{x}_p$ has been used for actually updating the weight vector at some point during learning
- At that time, the weights are updated as follows

$$\mathbf{w} = \sum_p N_p [\eta (d_p - y_p) \mathbf{x}_p]$$

Recall $[d-y]$ is either 0 (correct), -2 (too strong), or 2 (too weak)

If $d = 1$, $(d-y)$ is 2

If $d = -1$, $(d-y)$ is -2

$$= 2\eta \sum_p N_p d_p \mathbf{x}_p$$

$$d_p \times 2 = -2 \text{ when } (d = -1)$$

$$d_p \times 2 = 2 \text{ when } (d = 1)$$

Proof (cont.)

Perceptron Convergence Theorem

- When updating the weights: $\mathbf{w} = 2\eta \sum_p N_p d_p \mathbf{x}_p$

- Consider $\mathbf{w}_o^T \mathbf{w}$ $\mathbf{w}_o^T \mathbf{w} = 2\eta \sum_p N_p d_p \mathbf{w}_o^T \mathbf{x}_p$

Recall that

$$d_p \mathbf{w}_o^T \mathbf{x}_p \geq \alpha > 0$$

So $\mathbf{w}_o^T \mathbf{w} \geq 2\eta\alpha \sum_p N_p = 2\eta\alpha P$ (2) where $P = \sum_p N_p$

From (1)

Proof (cont.)

Perceptron Convergence Theorem

$$\begin{aligned}\mathbf{w}(n+1) &= \mathbf{w}(n) + \eta[d(n) - y(n)]\mathbf{x}(n) \\ &= \mathbf{w}(n) + 2\eta d(n)\mathbf{x}(n) \\ &= \mathbf{w}(n) + \Delta\mathbf{w}\end{aligned}$$

- Now consider the change in square length, $\|\mathbf{w}\|$, *after a single update by $\mathbf{x}$*

$$\begin{aligned}\Delta\|\mathbf{w}\|^2 &= \|\mathbf{w} + \Delta\mathbf{w}\|^2 - \|\mathbf{w}\|^2 \\ &= \|\mathbf{w} + 2\eta d\mathbf{x}\|^2 - \|\mathbf{w}\|^2 \\ &= 4\eta^2\|\mathbf{x}\|^2 + 4\eta d\mathbf{w}^T\mathbf{x} \\ &\leq 4\eta^2\beta \quad \text{where } \beta = \max_p \|\mathbf{x}_p\|^2\end{aligned}$$

Since upon an update, $d\mathbf{w}^T\mathbf{x} \leq 0$

Proof (cont.)

Perceptron Convergence Theorem

$$\mathbf{w}_o^T \mathbf{w} \geq 2\eta\alpha \sum_p N_p = 2\eta\alpha P \quad (2)$$

- By summing $\|\mathbf{w}\|^2$ for P steps we have the bound:

$$\|\mathbf{w}\|^2 \leq 4\eta^2 \beta P \quad (3)$$

- Now square the cosine of the angle between $\mathbf{w}_o$ and $\mathbf{w}$, we have by (2) and (3)

$$1 \geq \frac{(\mathbf{w}_o^T \mathbf{w})^2}{\|\mathbf{w}_o\|^2 \|\mathbf{w}\|^2} \geq \frac{4\eta^2 \alpha^2 P^2}{4\eta^2 \beta P \|\mathbf{w}_o\|^2} = \frac{\alpha^2 P}{\beta \|\mathbf{w}_o\|^2}$$

Cauchy-Schwarz
Inequality

P must be finite to satisfy this inequality

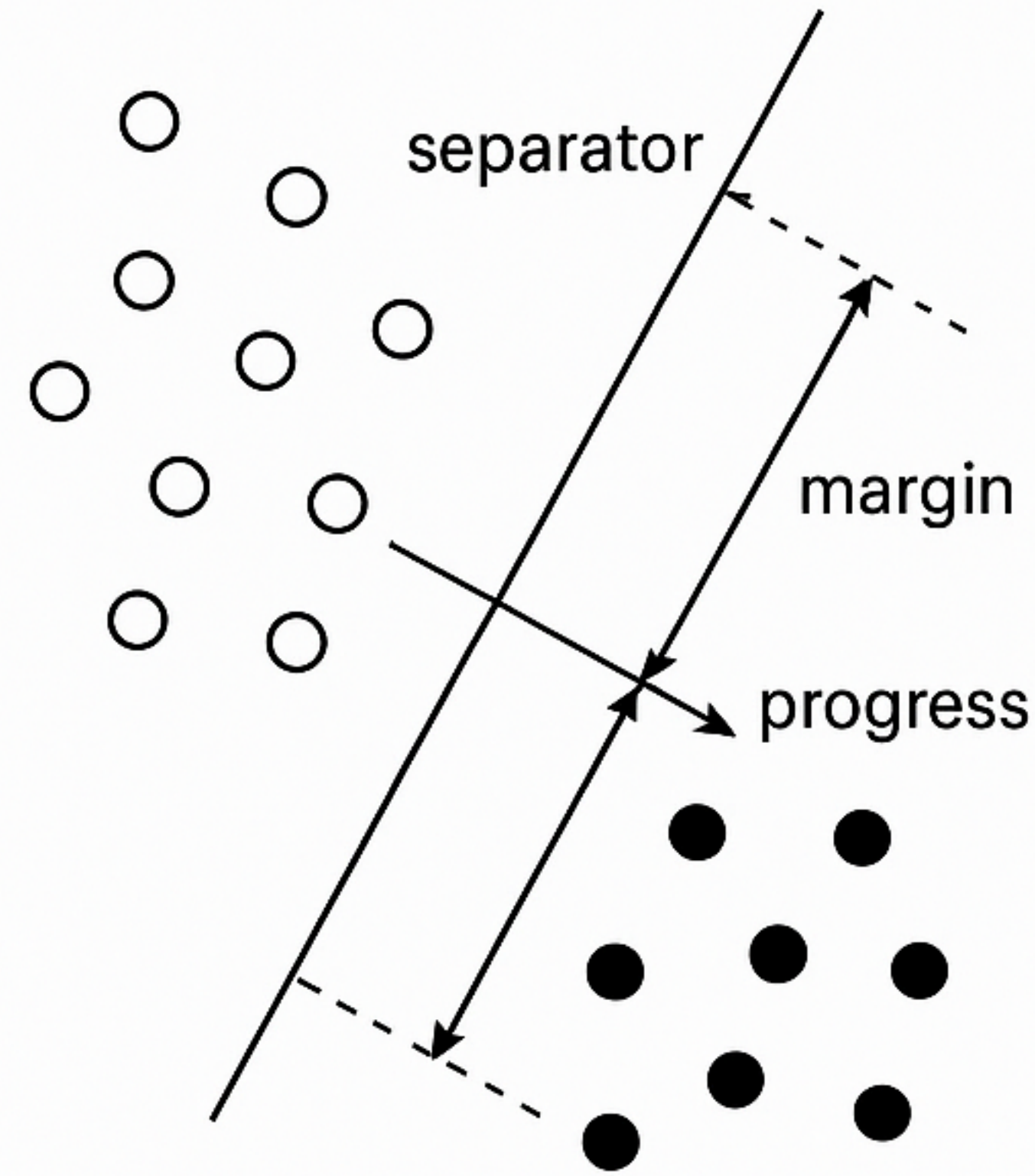
Proof outline

Block–Novikoff Perceptron Mistake Bound

- **Setting:** Training data is linearly separable with a margin (a gap between positive and negative examples).
- **Key idea:** Each mistake by the perceptron both
 - pushes the weight vector more in the *right* direction, and
 - increases its length only by a limited amount.
- **Proof strategy:**
 - Track how much “progress” the weight vector makes toward the ideal separator.
 - Compare this progress to how quickly its length can grow.
- **Result:** The perceptron can only make a bounded number of mistakes before it aligns well enough with the separator.
- **Bound:** Mistakes $\leq (\text{radius of data} / \text{margin})^2$.
- **Takeaway:** The perceptron always converges on separable data, and the number of mistakes depends only on geometry (margin + spread), **not on dimension**.

Block–Novikoff Perceptron Mistake Bound

- Setting: Training data is linearly separable with a margin (a gap between positive and negative examples)
- Key idea: Each mistake by the perceptron both pushes the weight vector more in the *right* direction, and increases its length only by a limited amount
- Proof strategy: Track how much “progress” the weight vector makes toward the ideal separator
 - Compare its progress to how quickly its length can grow
- Result: The perceptron can only make a bounded number of mistakes before it aligns well enough with the separator
- Takeaway: The perceptron always converges on separable data, and the number of mistakes depends only on geometry (margin+spread)



Alternate sources for proof

- 3 page note by Collins in week 3 module on Carmen
- Freund and Schapire paper

Practicalities

- If dataset is not linearly separable, algorithm will loop => impose limit on iterations
- If separable, solution guaranteed, but not guaranteed to be the best generalizer
 - Use averaged or voted perceptron. Run for a bit, then combine weights (Freund and Schapire, see paper in Week 3)
 - Use a regularized linear model